Object Oriented Programming (CSEG1044)

Unit 04 – Lecture 01: Generics Fundamentals (Generic Classes)

April 1, 2026

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Quick Links

Overview Key Topics Demo Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Agenda

1 Overview

2 Key Topics

3 Demo / Example

4 Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Generics, Lambdas, Swing, and JDBC

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Generics, Lambdas, Swing, and JDBC
- **Course thread:** Use Generics Fundamentals (Generic Classes) to move Campus Resource Manager toward a real desktop application with UI, callbacks, and persistence.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Recall Before You Start

- **Classes and methods**: A generic class is still a class; generics only add type parameters to familiar structures. Main reference: `Unit 02 / Lecture 01 slides/notes`.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Recall Before You Start

- **Classes and methods**: A generic class is still a class; generics only add type parameters to familiar structures. Main reference: `Unit 02 / Lecture 01 slides/notes`.
- **Type relationships**: Generic type safety is easier to read once basic inheritance intuition is already clear. Main reference: `Unit 02 / Lecture 04 slides/notes`.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Learning Outcomes

- Explain why generic classes are better than duplicated classes or `Object`-only designs.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Learning Outcomes

- Explain why generic classes are better than duplicated classes or `Object`-only designs.
- Read and write a generic class such as `ResourceHolder<T>`.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Learning Outcomes

- Explain why generic classes are better than duplicated classes or `Object`-only designs.
- Read and write a generic class such as `ResourceHolder<T>`.
- Distinguish **type parameter**, **type argument**, and **parameterized type**.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Learning Outcomes

- Explain why generic classes are better than duplicated classes or `Object`-only designs.
- Read and write a generic class such as `ResourceHolder<T>`.
- Distinguish **type parameter**, **type argument**, and **parameterized type**.
- Identify raw-type usage and explain why it weakens type safety.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Why this topic exists* → *Attempt 1: separate classes for every type* → *Attempt 2: use one class with `Object`* → *Why `Object` is unsafe*.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Why this topic exists* → *Attempt 1: separate classes for every type* → *Attempt 2: use one class with `Object`* → *Why `Object` is unsafe*.
3. Trace the worked example and connect each line back to one concept frame.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Why this topic exists* → *Attempt 1: separate classes for every type* → *Attempt 2: use one class with `Object`* → *Why `Object` is unsafe*.
3. Trace the worked example and connect each line back to one concept frame.
4. Attempt the mini exercises before moving to the recap and exit check.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Unit 04: Generics, Lambdas, GUI Swing & Database Connectivity

- Lecture focus: Generics Fundamentals (Generic Classes)

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Today: Teaching Path

1. Same problem solved with many separate classes

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Today: Teaching Path

1. Same problem solved with many separate classes
2. Same problem solved with one `Object`-based class

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Today: Teaching Path

1. Same problem solved with many separate classes
2. Same problem solved with one `Object`-based class
3. Same problem solved with one generic class

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Today: Teaching Path

1. Same problem solved with many separate classes
2. Same problem solved with one `Object`-based class
3. Same problem solved with one generic class
4. Correct usage, raw-type mistakes, and a small project connection

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Start with the problem

- Suppose the project stores different campus resources: books, laptops, and lab keys.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Start with the problem

- Suppose the project stores different campus resources: books, laptops, and lab keys.
- A beginner often creates a separate holder class for each type.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Start with the problem

- Suppose the project stores different campus resources: books, laptops, and lab keys.
- A beginner often creates a separate holder class for each type.
- That works, but the class logic becomes repeated with only the data type changing.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Repeated classes: works, but duplicates logic

```java
class BookHolder {
    private Book item;
    void set(Book item) {
        this.item = item;
    }
    Book get() {
        return item;
    }
}
```

```java
class LaptopHolder {
    private Laptop item;
    void set(Laptop item) {
        this.item = item;
    }
    Laptop get() {
        return item;
    }
}
```

- Structure is the same; only the type changes.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

The first improvement many students try

- "Why not store everything as `Object`?"

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

The first improvement many students try

- "Why not store everything as `Object`?"
- This removes duplication because one class can hold any reference type.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

The first improvement many students try

- "Why not store everything as `Object`?"
- This removes duplication because one class can hold any reference type.
- But now the compiler no longer knows the real type stored inside.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

One Object-based class: less duplication, more risk

```
class ResourceHolder {
    private Object item;

    void set(Object item) { this.item = item; }
    Object get() { return item; }
}
```

- You can store a Book, Laptop, or LabKey.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

One Object-based class: less duplication, more risk

```
class ResourceHolder {
    private Object item;

    void set(Object item) { this.item = item; }
    Object get() { return item; }
}
```

- You can store a Book, Laptop, or LabKey.
- But get() returns Object, so the caller must cast.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., java.util.function, javax.swing, java.sql) (accessed April 1, 2026).

Why Object becomes unsafe

```
ResourceHolder holder = new ResourceHolder();
holder.set(new Laptop("L-14"));

Book b = (Book) holder.get(); // compiles
```

- The cast compiles.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Why Object becomes unsafe

```
ResourceHolder holder = new ResourceHolder();
holder.set(new Laptop("L-14"));

Book b = (Book) holder.get(); // compiles
```

- The cast compiles.
- The data is actually a Laptop, not a Book.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Why Object becomes unsafe

```
ResourceHolder holder = new ResourceHolder();
holder.set(new Laptop("L-14"));

Book b = (Book) holder.get(); // compiles
```

- The cast compiles.
- The data is actually a Laptop, not a Book.
- Error appears at runtime as ClassCastException.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Generics solve both sides of the problem

- We still want **one reusable class**.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Generics solve both sides of the problem

- We still want **one reusable class**.
- We also want **compile-time type checking**.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Generics solve both sides of the problem

- We still want **one reusable class**.
- We also want **compile-time type checking**.
- A generic class gives both: reuse *and* safety.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Generic class design

```
class ResourceHolder<T> {
    private T item;

    void set(T item) { this.item = item; }
    T get() { return item; }
}
```

- T is a type parameter.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Generic class design

```
class ResourceHolder<T> {
    private T item;

    void set(T item) { this.item = item; }
    T get() { return item; }
}
```

- T is a type parameter.
- Inside the class, T behaves like "the type we will decide later".

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

How to read the syntax

- `class ResourceHolder<T>` means "define a class that works for some type `T`".

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

How to read the syntax

- `class ResourceHolder<T>` means "define a class that works for some type `T`".
- `ResourceHolder<Book>` means "this particular holder stores `Book` objects".

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

How to read the syntax

- `class ResourceHolder<T>` means "define a class that works for some type `T`".
- `ResourceHolder<Book>` means "this particular holder stores `Book` objects".
- `T` is the **type parameter**; `Book` is the **type argument**.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

How to read the syntax

- `class ResourceHolder<T>` means "define a class that works for some type `T`".
- `ResourceHolder<Book>` means "this particular holder stores `Book` objects".
- `T` is the **type parameter**; `Book` is the **type argument**.
- After substitution, `ResourceHolder<Book>` is a **parameterized type**.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Using the same class with different resource types

```
ResourceHolder<Book> bookHolder = new ResourceHolder<>();
ResourceHolder<Laptop> laptopHolder = new ResourceHolder
    <>();
ResourceHolder<LabKey> keyHolder = new ResourceHolder<>();
```

- Same class definition

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Using the same class with different resource types

```
ResourceHolder<Book> bookHolder = new ResourceHolder<>();
ResourceHolder<Laptop> laptopHolder = new ResourceHolder
    <>();
ResourceHolder<LabKey> keyHolder = new ResourceHolder<>();
```

- Same class definition
- Different type-safe usages

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Using the same class with different resource types

```
ResourceHolder<Book> bookHolder = new ResourceHolder<>();
ResourceHolder<Laptop> laptopHolder = new ResourceHolder
    <>();
ResourceHolder<LabKey> keyHolder = new ResourceHolder<>();
```

- Same class definition
- Different type-safe usages
- No repeated holder classes

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

What the compiler now protects

- A `ResourceHolder<Book>` accepts `Book` values, not `Laptop` values.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

What the compiler now protects

- A `ResourceHolder<Book>` accepts `Book` values, not `Laptop` values.
- `get()` returns `Book`, so no cast is needed.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

What the compiler now protects

- A `ResourceHolder<Book>` accepts `Book` values, not `Laptop` values.
- `get()` returns `Book`, so no cast is needed.
- Many mistakes move from runtime to compile time.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Important rule: primitives are not type arguments

- `ResourceHolder<int>` is invalid.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Important rule: primitives are not type arguments

- `ResourceHolder<int>` is invalid.
- Generics need reference types.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Important rule: primitives are not type arguments

- `ResourceHolder<int>` is invalid.
- Generics need reference types.
- Use wrapper classes such as `Integer`, `Double`, and `Character`.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Raw types: old style, unsafe style

- Raw type means writing `ResourceHolder` instead of `ResourceHolder<Book>` or another parameterized type.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Raw types: old style, unsafe style

- Raw type means writing `ResourceHolder` instead of `ResourceHolder<Book>` or another parameterized type.
- Java allows it for backward compatibility with pre-generics code.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Raw types: old style, unsafe style

- Raw type means writing `ResourceHolder` instead of `ResourceHolder<Book>` or another parameterized type.
- Java allows it for backward compatibility with pre-generics code.
- But you lose most compile-time type checking.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Raw types: old style, unsafe style

- Raw type means writing `ResourceHolder` instead of `ResourceHolder<Book>` or another parameterized type.
- Java allows it for backward compatibility with pre-generics code.
- But you lose most compile-time type checking.
- In modern code, warnings about raw types should usually be treated as design problems.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Where you will meet generic classes again

- Java library classes use the same idea: collections, maps, queues, optional wrappers, and more.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Where you will meet generic classes again

- Java library classes use the same idea: collections, maps, queues, optional wrappers, and more.
- In your own project, generic classes are useful for reusable containers, response wrappers, repositories, and helper utilities.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Where you will meet generic classes again

- Java library classes use the same idea: collections, maps, queues, optional wrappers, and more.
- In your own project, generic classes are useful for reusable containers, response wrappers, repositories, and helper utilities.
- This lecture only builds the foundation; the next lecture extends it to methods, bounds, and wildcards.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Checkpoint and Reflection

Reflection prompt

Where would *Generics Fundamentals (Generic Classes)* matter most in Campus Resource Manager, and which design choice would you justify using this lecture's ideas?

Quick self-check

Which part needs one more example before you move on: *Why this topic exists*, *Attempt 1: separate classes for every type*, the worked example, or the recap?

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Worked example: sample domain classes

```
class Book {
    String title;
    Book(String title) { this.title = title; }
}

class LabKey {
    String roomCode;
    LabKey(String roomCode) { this.roomCode = roomCode; }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Worked example: generic holder

```
class ResourceHolder<T> {
    private final T item;
    ResourceHolder(T item) { this.item = item; }
    T get() { return item; }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Safe usage: one class, different types

```
public class DemoSafe {
    public static void main(String[] args) {
        ResourceHolder<Book> b =
            new ResourceHolder<>(new Book("Clean Code"));
        ResourceHolder<LabKey> k =
            new ResourceHolder<>(new LabKey("LAB-2"));

        Book book = b.get();
        LabKey key = k.get();
        System.out.println(book.title + " / " + key.
            roomCode);
    }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Unsafe usage: raw type removes the benefit

```
public class DemoRaw {
    public static void main(String[] args) {
        ResourceHolder raw =
            new ResourceHolder(new LabKey("LAB-2"));

        Book wrong = (Book) raw.get(); // runtime risk
        System.out.println(wrong.title);
    }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Why is a single `Object`-based class better than duplicated classes, but still not the best final design?

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Why is a single `Object`-based class better than duplicated classes, but still not the best final design?
2. What is the difference between `T` in `ResourceHolder<T>` and `Book` in `ResourceHolder<Book>`?

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Why is a single `Object`-based class better than duplicated classes, but still not the best final design?
2. What is the difference between `T` in `ResourceHolder<T>` and `Book` in `ResourceHolder<Book>`?
3. Why do we write `ResourceHolder<Integer>` instead of `ResourceHolder<int>`?

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Watch-outs

- Treating generic type parameters as if they were inheritance declarations.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Watch-outs

- Treating generic type parameters as if they were inheritance declarations.
- Falling back to raw types and losing type safety.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Watch-outs

- Treating generic type parameters as if they were inheritance declarations.
- Falling back to raw types and losing type safety.
- Expecting primitive types to work directly as generic arguments.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Key Takeaways

- Separate classes for each type cause duplication.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Key Takeaways

- Separate classes for each type cause duplication.
- A single `Object`-based class removes duplication but pushes errors to runtime.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Key Takeaways

- Separate classes for each type cause duplication.
- A single `Object`-based class removes duplication but pushes errors to runtime.
- A generic class such as `ResourceHolder<T>` gives reuse with compile-time checking.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Key Takeaways

- Separate classes for each type cause duplication.
- A single `Object`-based class removes duplication but pushes errors to runtime.
- A generic class such as `ResourceHolder<T>` gives reuse with compile-time checking.
- Raw types weaken the protection that generics are meant to provide.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Checkpoint Questions

1. What design problem leads us toward generic classes?

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Checkpoint Questions

1. What design problem leads us toward generic classes?
2. What is the meaning of `ResourceHolder<Book>`?

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Checkpoint Questions

1. What design problem leads us toward generic classes?
2. What is the meaning of `ResourceHolder<Book>`?
3. Why is raw `ResourceHolder` unsafe compared with `ResourceHolder<Book>`?

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.
- Apply the idea once inside Campus Resource Manager to make the concept stick.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

Next Lecture Preview

Next: Generic methods, bounded type parameters, and wildcards

- We will move from generic classes to methods and then learn how to restrict acceptable types.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).